

Hunter College
Department of Computer Science
CSCI 39548: Practical Web Development
Summer 2026
Final Project — Campuses & Students (Fullstack CRUD)
Instructor: *Sourya Saha* (sourya.saha24@login.cuny.edu)

Weight	40% of course grade
Introduced	Monday, June 1, 2026 (Class 1)
Group formation begins	Class 1 (via GitHub)
Groups finalized	Wednesday, June 10, 2026
Live demos + code due	Monday, July 6, 2026
Submission	Brightspace — one submission per team: GitHub repo URLs (client + server) + deployed URLs (frontend + backend)
Type	Team of 2–3 (no solo, no teams of 4+)

1 Overview

Build and deploy a **fullstack relational CRUD web application** for managing **Campuses** and the **Students** enrolled at them. You will build this **incrementally over the entire semester** — each module you learn (HTML/CSS, React, Router, TanStack Query, Tailwind, Zustand, Express, Prisma, deploy) maps directly to a feature of the project. **The course is the project build.**

You'll work in **teams of 2 or 3**, using a real Git/GitHub collaboration workflow (feature branches, pull requests, code review). The same workflow you practiced solo in A1 — now multi-person, for real.

Note: This is the largest grade item in the course (40%). It is also the most rewarding: a deployed, end-to-end web app you and your team built, on a public URL you can show people.

2 The Domain — Campuses $\leftrightarrow$ Students

A **Campus** has many **Students**. A **Student** belongs to **at most one** Campus (a Student may also be **unenrolled** — i.e., have no Campus).

2.1 Campus Fields

- `id`
- `name`
- `address`
- `imageUrl` (a default image if none is given)
- `description`

2.2 Student Fields

- `id`
- `firstName`
- `lastName`
- `email`
- `imageUrl` (a default image if none is given)
- `gpa` (a decimal between 0.0 and 4.0)
- `campusId` (nullable — unenrolled students are allowed)

You may add additional fields, but the ones above are required.

3 Required Features

3.1 Pages and Routing (Client)

- **Home page** with project title and links into the app.

- **All Campuses** — a list view of every campus (image + name + link).
- **Single Campus** — campus details + the list of enrolled students.
- **All Students** — a list view of every student (image + name + link).
- **Single Student** — student details + the campus they are enrolled at (or “Not enrolled” if `campusId` is null).
- **Add Campus** — form to create a new campus.
- **Add Student** — form to create a new student (with optional campus selection).
- **Edit Campus / Edit Student** — forms to update an existing record.
- **404 page** for unmatched routes.

Routing must use **React Router**. The navigation bar must use `NavLink` with an active-state style.

3.2 CRUD Operations

- **Create, Read, Update, Delete** — for **both** Campuses **and** Students.
- Deleting a Campus must **not** delete its Students — they become unenrolled instead.
- From the Single Campus page, you can **remove a student** from that campus (un-enroll).
- From the Single Student page, you can **change** the student’s campus (or set it to none).

3.3 State and Data

- **TanStack Query** is used for **all** server data (fetch, cache, invalidate on mutation). No `useEffect(fetch())` in components.
- **Zustand** is used for at least one piece of **client-only state** (e.g. dark mode, filter, sidebar open/closed, recently viewed).

- **Forms** are validated client-side (required fields, valid email format, GPA between 0 and 4).
- **Loading and error states** are shown for every fetched view.

3.4 Styling

- All styling is done with **Tailwind CSS**.
- The app is **responsive** — usable on phone and desktop widths.
- A consistent visual style across pages (shared layout/header/footer).

3.5 Backend (Server)

- **Node + Express** REST API.
- **Prisma** as the ORM, **PostgreSQL** as the database (Neon free tier).
- Endpoints for full CRUD on both resources:
 - GET, POST, PUT, DELETE for `/campuses` and `/campuses/:id`
 - The same for `/students` and `/students/:id`
- **Server-side validation** — never trust the client. Reject malformed input with a 400.
- **CORS** configured so the deployed frontend can call the deployed backend.
- A **seed script** that populates the DB with a handful of campuses and students so the deployed app isn't empty.

3.6 Deploy

- **Frontend** deployed to **Vercel** (or Netlify).
- **Backend** deployed to **Render** (or Railway).
- **Database** on **Neon** (Postgres).
- Both deployed URLs are live and working at submission time.

3.7 Git/GitHub Workflow (Team)

- **Two public repositories** — one for the client, one for the server. (A monorepo is also acceptable — talk to me first.)
- Every team member has **commits in their own name**.
- Work happens on **feature branches**, merged via **pull requests** — **not** pushed straight to **main**.
- Each PR is **reviewed by at least one other team member** before merge.
- **README.md** in each repo explains the project, the stack, how to run it locally, and the deployed URLs.

4 Optional Bonus — Authentication (Clerk)

Adding **Clerk** authentication so that only signed-in users can create/edit/delete data (read remains public) is worth **+5 bonus points** (cap at 100). This is covered lightly in the final modules; you'll mostly self-study from the demo + Clerk docs.

5 Team Formation

- Teams of **2 or 3**. **No solo, no teams of 4 or more**.
- Start finding teammates **Class 1**. Use Brightspace + the class chat.
- Finalize your team by **Wednesday, June 10** and tell me the roster + repo URLs.
- **Every team member contributes code** that is visible in `git log`. A teammate who only watches will be graded individually downward.

6 How the Project Is Built — Incrementally

You don't build this in the last week. You build it as you learn:

After learning...	What you can add to the project
HTML/CSS	Static pages, layout, styles for Home / All Campuses
JavaScript / TypeScript	Form validation logic
React core	Componentize, <code>useState</code> , controlled forms, lifting state
React + Tailwind	Hooks polish, real Tailwind styling
Router + TanStack Query	Real routing for all pages, fetch/cache
Zustand + Express	Client state, scaffold Express backend
Prisma + deploy	Wire up Prisma + Postgres, deploy both halves

Your stack should be complete and deployed before demo day. **Monday, July 6** is both demo day and the code deadline.

7 Submission

Submit on **Brightspace** — **one submission per team** (any member can submit; the others are listed in the submission text). Paste:

1. **Client repo URL** — e.g.
`https://github.com/teamname/campuses-client`
2. **Server repo URL** — e.g.
`https://github.com/teamname/campuses-server`
3. **Deployed frontend URL** — e.g.
`https://campuses-team.vercel.app`
4. **Deployed backend URL** — e.g.
`https://campuses-api.onrender.com`
5. **Team roster** — each member's full name + GitHub username.

Make sure **both repos are public**, **both deployments are live**, and the deployed frontend talks to the deployed backend. Test in a private/incognito window before submitting.

8 Demo Day — Monday, July 6

Each team gives a ~8–10 minute live demo of the deployed app:

1. **Walk through the app** — show every required page and CRUD operation working against the deployed backend.
2. **Show one piece of code** your team is proud of (a tricky query, a clever component, a tough bug you fixed).
3. **Each team member speaks** about what they contributed.

Slides are optional. The deployed app is the centerpiece.

9 Grading Rubric (100 pts + 5 bonus)

Criterion	Pts
CRUD completeness — all pages, all operations, both resources	20
Relational behavior — enrolment / un-enrolment / unenrolled students handled correctly	10
Backend quality — Express routes, Prisma schema, validation, seed script	15
Frontend data layer — TanStack Query (cache + invalidation), Zustand client state	10
Routing & navigation — Router setup, NavLink, 404, no broken links	8
Tailwind styling & responsiveness — coherent design, works on mobile + desktop	10
Deployment — both halves live and talking to each other at submission time	10
Team Git workflow — feature branches + PRs + reviews + everyone has commits	10
Demo quality — clear walkthrough, every member speaks, deployed app used	7
Optional: Clerk auth	+5

10 Notes

- **Individual grading.** The project grade is per team, but I reserve the right to adjust **individual** grades up or down based on `git log`, PR authorship, and demo participation. Teammates who don't contribute will be graded accordingly.
- **Late policy.** The code deadline (Mon Jul 6) is firm because of grade submission deadlines. Plan accordingly.
- **Use of AI.** AI is welcome (see syllabus), but **you must be able to explain every part of your submission** during the demo if asked.

- **Stuck?** Discuss with your teammates first, then look at web resources (MDN, Stack Overflow, official docs). If you're still stuck, bring questions to class or email to set up office hours. Don't wait until the last week — the project rewards consistent weekly progress.
- **Starter repos** (client + server scaffolds) will be provided before the backend modules begin (around Class 8). Until then, your project repo is your own scaffold — start with what you have and refactor as you learn.

Final Project at a Glance

What: A deployed Campuses & Students CRUD app, built by a team of 2–3.

Stack: React + Router + TanStack Query + Tailwind + Zustand · Express + Prisma + Postgres (Neon) · Vercel + Render.

Cadence: Build a little every week as we learn each module.

Demo + Code due: Mon Jul 6.

*End of **Final Project***